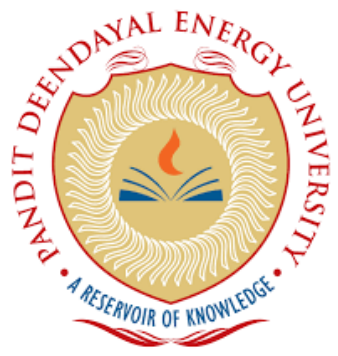




IoT

Communication and Framework

Dr Abhishek Kumar

- SPI
- I2C
- Introduction to Simulators
 - <https://azure-samples.github.io/raspberry-pi-web-simulator/#getstarted>
 - <https://docs.wokwi.com/>
 - <https://create.withcode.uk/python/A3>
 - <https://beta.wyliodrin.studio/>
 - <https://fritzing.org/download/> [PAID]
- IoT Communication Technology, protocol

I²C vs SPI

- I²C ('Inter-Integrated Circuit', protocol) and
- SPI ('Serial Peripheral Interface')
- Both protocols are well suited for
 - communications between integrated circuits
 - for slow communication with on-board peripherals.
- Two major companies – Philips for I²C and Motorola for SPI

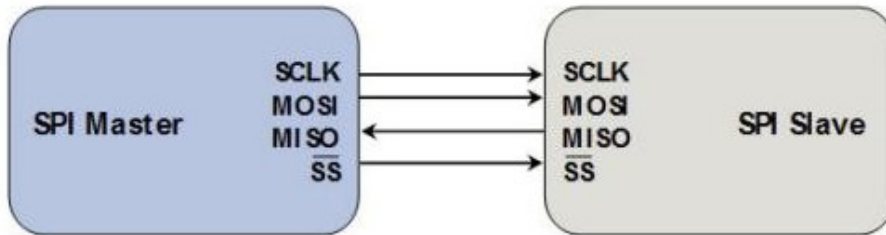
I2C and SPI: History

- I²C bus was developed in **1982**.
- Its purpose was to provide an easy way to connect a CPU to peripherals chips.
- Generally, peripheral devices in embedded systems are often connected to the microcontroller as memory-mapped I/O devices.
- In order to spare microcontroller pins, additional logic and make the PCBs simpler – in order words, to lower the costs – **Philips labs** in Netherlands invented the '**Inter-Integrated Circuit**', **IIC** or **I²C** protocol that only requires **2 wires** for connecting all the peripheral to a microcontroller.
- The original specification defined a bus speed of **100 kbps** (kilo bits per second).
- The specification was reviewed several times, introducing the **400 kbps speed in 1995** and – since 1998, **3.4 Mbps** for even faster peripherals.
- **Serial Peripheral Protocol (SPI)** was first introduced with the first microcontroller deriving from the same architecture as the popular **Motorola 68000 microprocessor in 1979**.

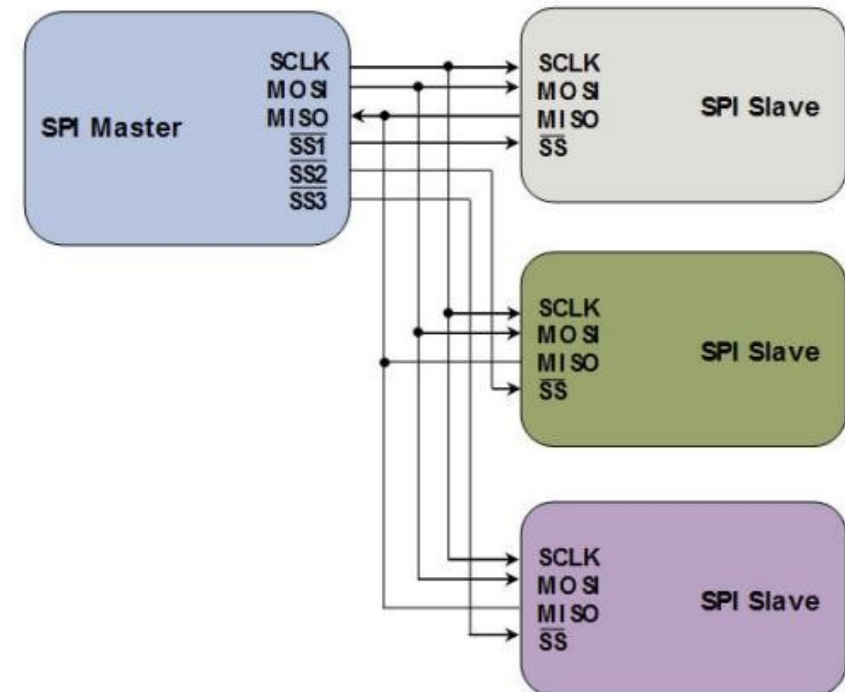
SPI

- SPI is a protocol on 4 signal lines

- A clock signal named **SCLK**, sent from the bus master to all slaves; all the SPI signals are synchronous to this clock signal;
- A slave select signal for each slave, **SS_n**, used to select the slave the master communicates with;
- A data line from the master to the slaves, named **MOSI** (Master Out-Slave In)
- A data line from the slaves to the master, named **MISO** (Master In-Slave Out).



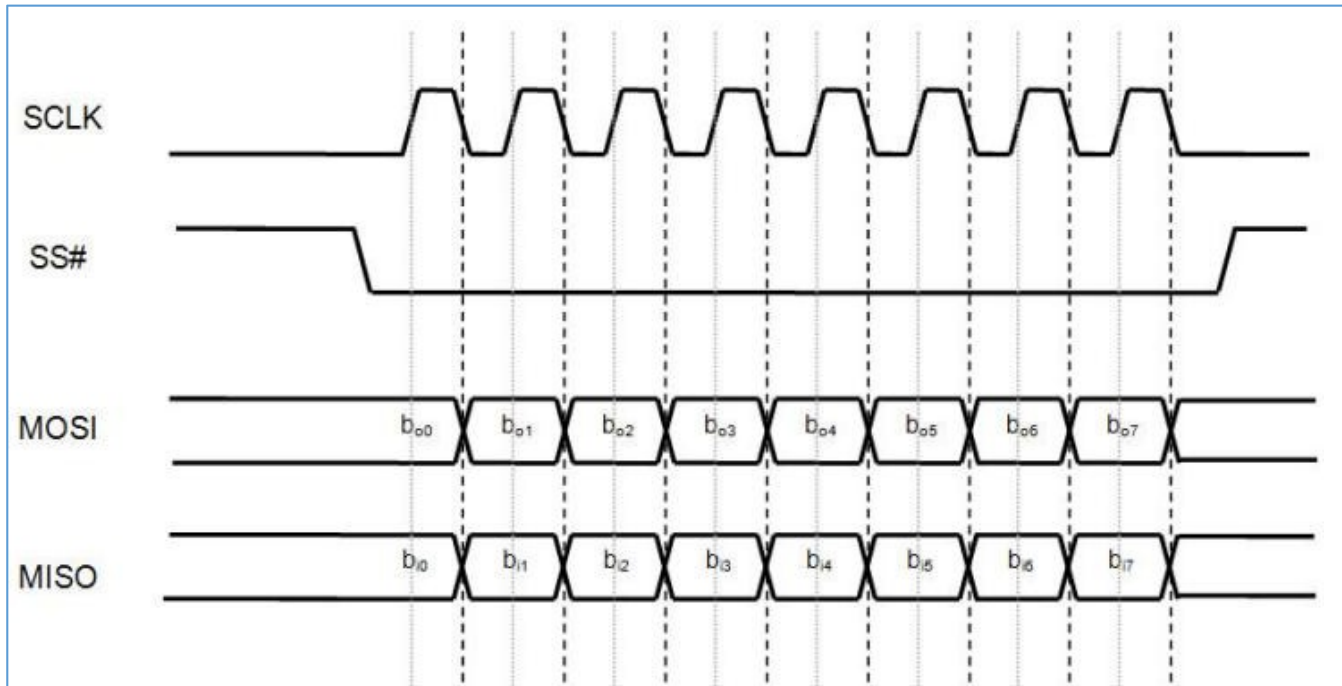
SPI master connected to a single slave
(point to point)



SPI master connected to a multiple slaves

SPI: Working

- SPI is a single-master communication protocol.
- This means that one central device initiates all the communications with the slaves.
- When the SPI master wishes to send data to a slave and/or request information from it, it selects slave by pulling the corresponding SS line low and it activates the clock signal at a clock frequency usable by the master and the slave.
- The master generates information onto MOSI line while it samples the MISO line



Data bits on MOSI and MISO toggle on the SCLK falling edge and are sampled on the SCLK rising edge.

The SPI mode defines which SCLK edge is used for toggling data and which SCLK edge is used for sampling data.

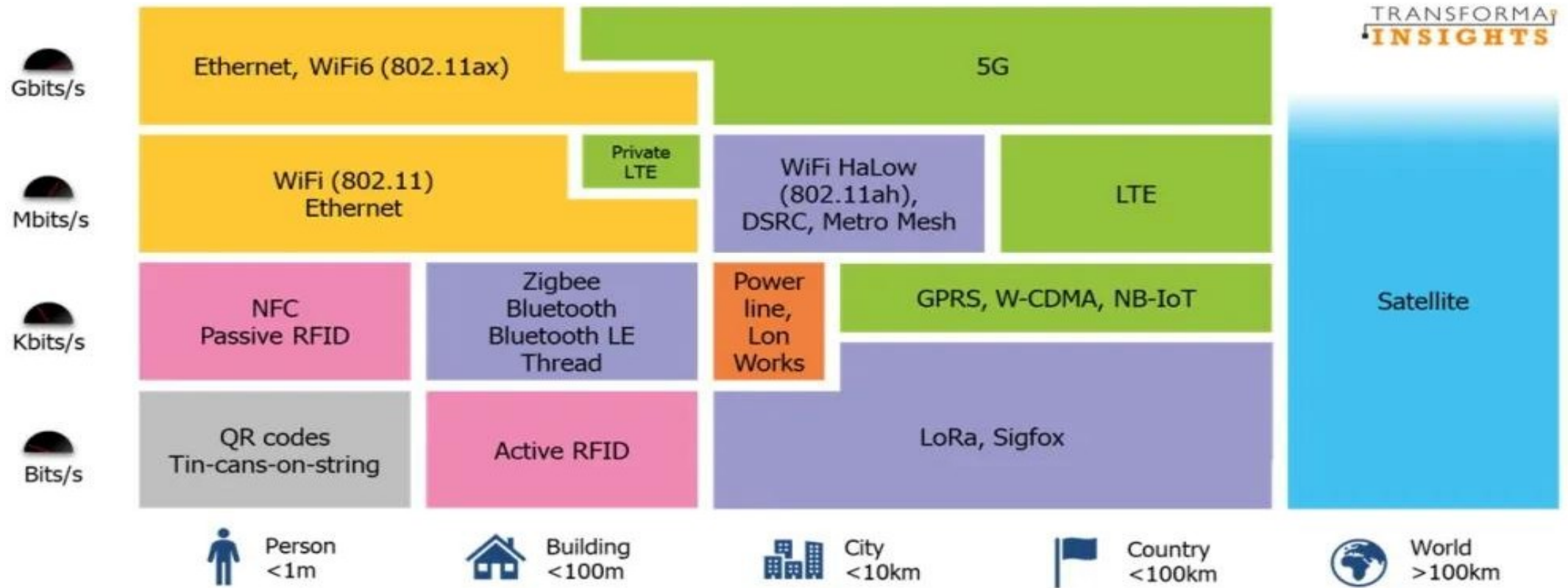
I2C

- I²C is a multi-master protocol that uses 2 signal lines.
- The two I²C signals are called '**serial data**' (SDA) and '**serial clock**' (SCL).
- There is **no need of chip select** (slave select) or arbitration logic.
- Virtually any number of slaves and any number of masters can be connected onto these 2 signal lines and communicate between each other using a protocol that defines:
 - **7-bits slave addresses: each device connected to the bus has got such a unique address;**
 - data divided into **8-bit bytes**
 - a few control bits for controlling the communication start, end, direction and for an acknowledgment mechanism
- The data rate has to be chosen **between 100 kbps (standard mode), 400 kbps (fast mode) and 3.4 Mbps (high speed mode)**
- Physically, the I²C bus consists of the 2 active wires SDA and SCL and a ground connection.
- The active wires are both bi-directional.
- The I2C protocol specification states that the IC that initiates a data transfer on the bus is considered the Bus Master. Consequently, at that time, all the other ICs are regarded to be Bus Slaves.

I2C : Working

- First, the master will issue a START condition.
- This acts as an 'Attention' signal to all of the connected devices.
- All ICs on the bus will listen to the bus for incoming data.
- Then the master sends the **ADDRESS** of the device it wants to access, along with an indication whether the access is a **Read or Write operation**.
- Having received the address, all IC's will compare it with their own address. If it doesn't match, they simply wait until the bus is released by the **stop condition**.
- However, If the address matches, the chip will produce a response called the **ACKNOWLEDGE** signal.
- Once the master receives the acknowledge, it can start transmitting or receiving **DATA**.
- When all is done, the master will issue the **STOP** condition.
- This is a signal that states the bus has been released and that the connected ICs may expect another transmission to start any moment.

IoT communication technologies: Overview

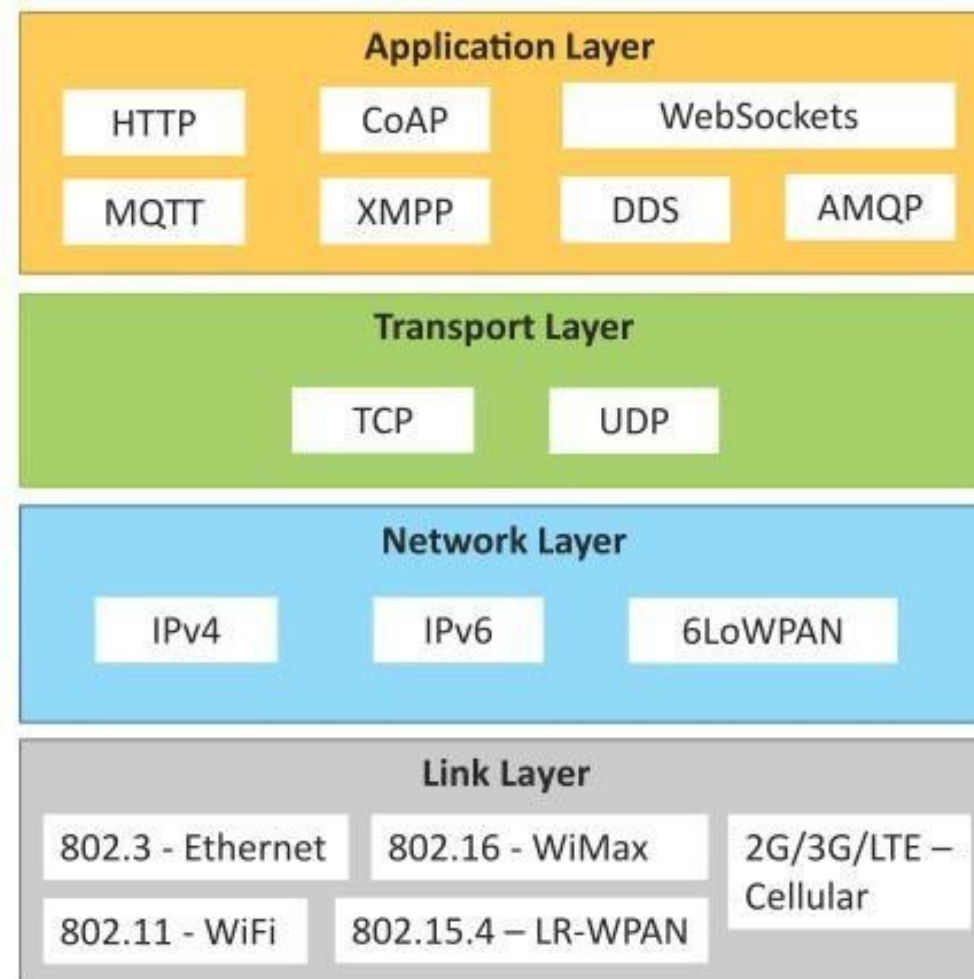


Review Paper: <https://www.mdpi.com/2248222>

ref: <https://www.embedded.com/the-internet-of-things-myth-the-search-for-a-connectivity-standard/>

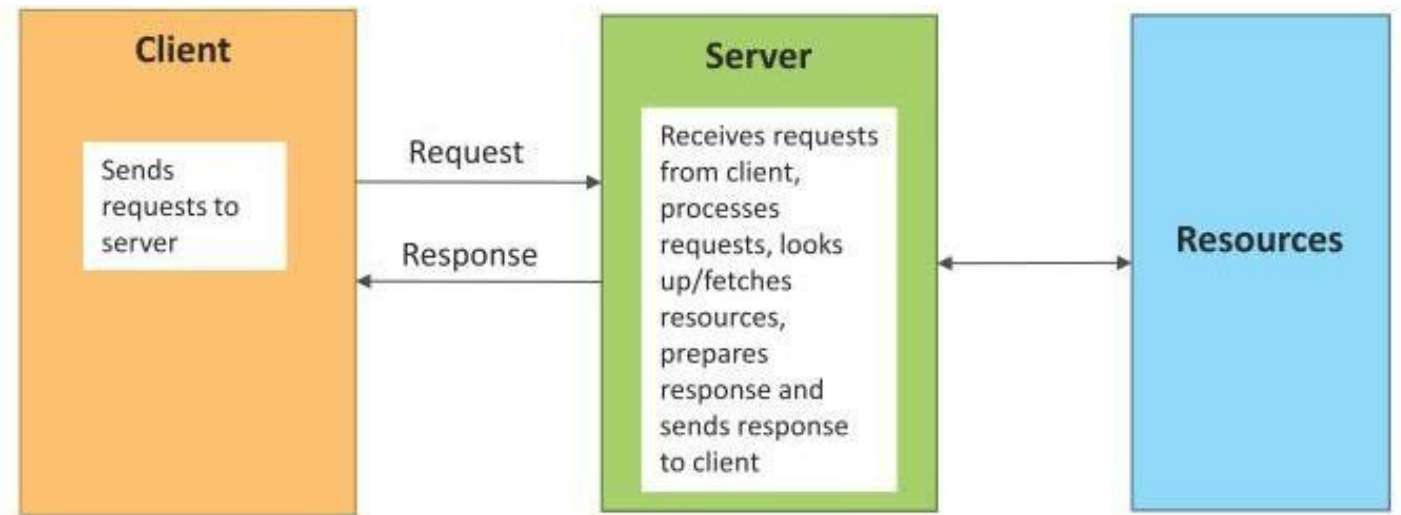
IoT Protocols

- Link Layer
 - 802.3 – Ethernet
 - 802.11 – WiFi
 - 802.16 – WiMax
 - 802.15.4 – LR-WPAN
 - 2G/3G/4G
- Network/Internet Layer
 - IPv4
 - IPv6
 - 6LoWPAN
- Transport Layer
 - TCP
 - UDP
- Application Layer
 - HTTP
 - CoAP
 - WebSocket
 - MQTT
 - XMPP
 - DDS
 - AMQP



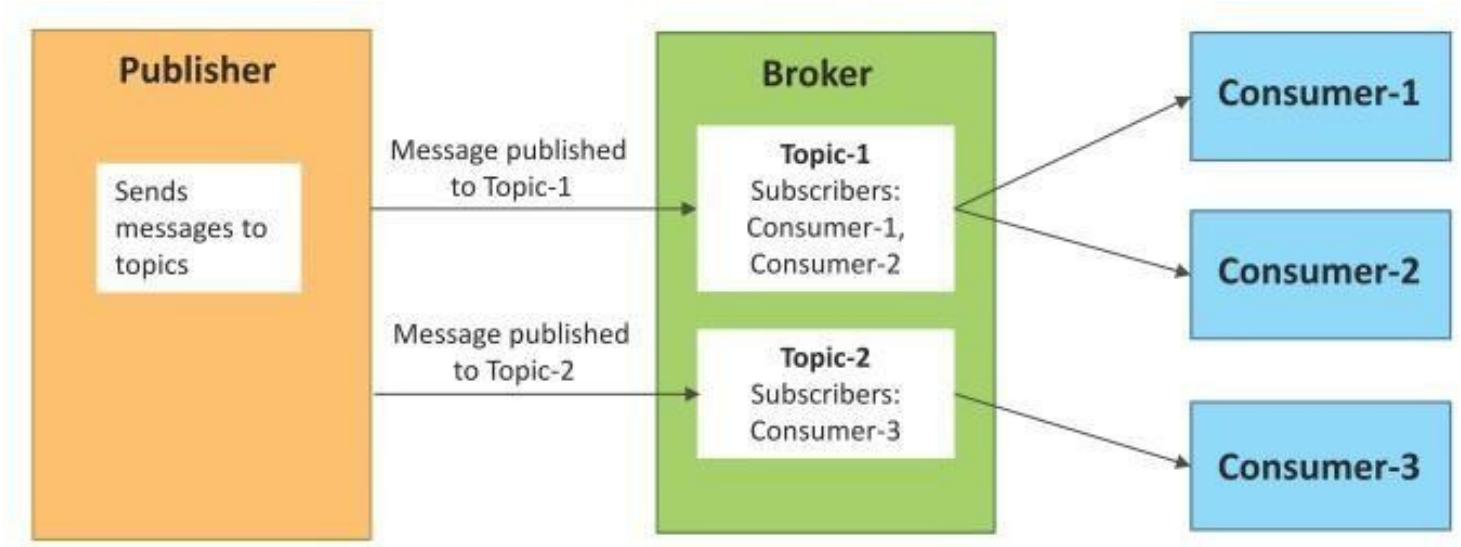
Request-Response communication model

- Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests.
- When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client.



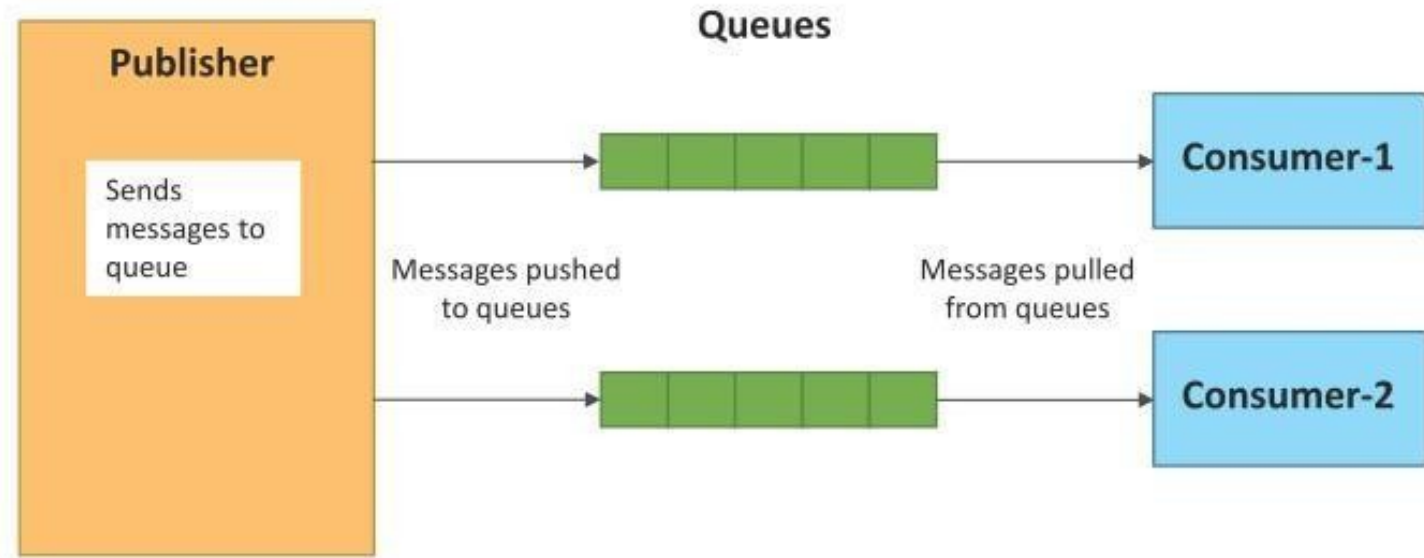
Publish-Subscribe communication model

- Publish-Subscribe is a communication model that involves publishers, brokers and consumers.
- Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers.
- Consumers subscribe to the topics which are managed by the broker.
- When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers.



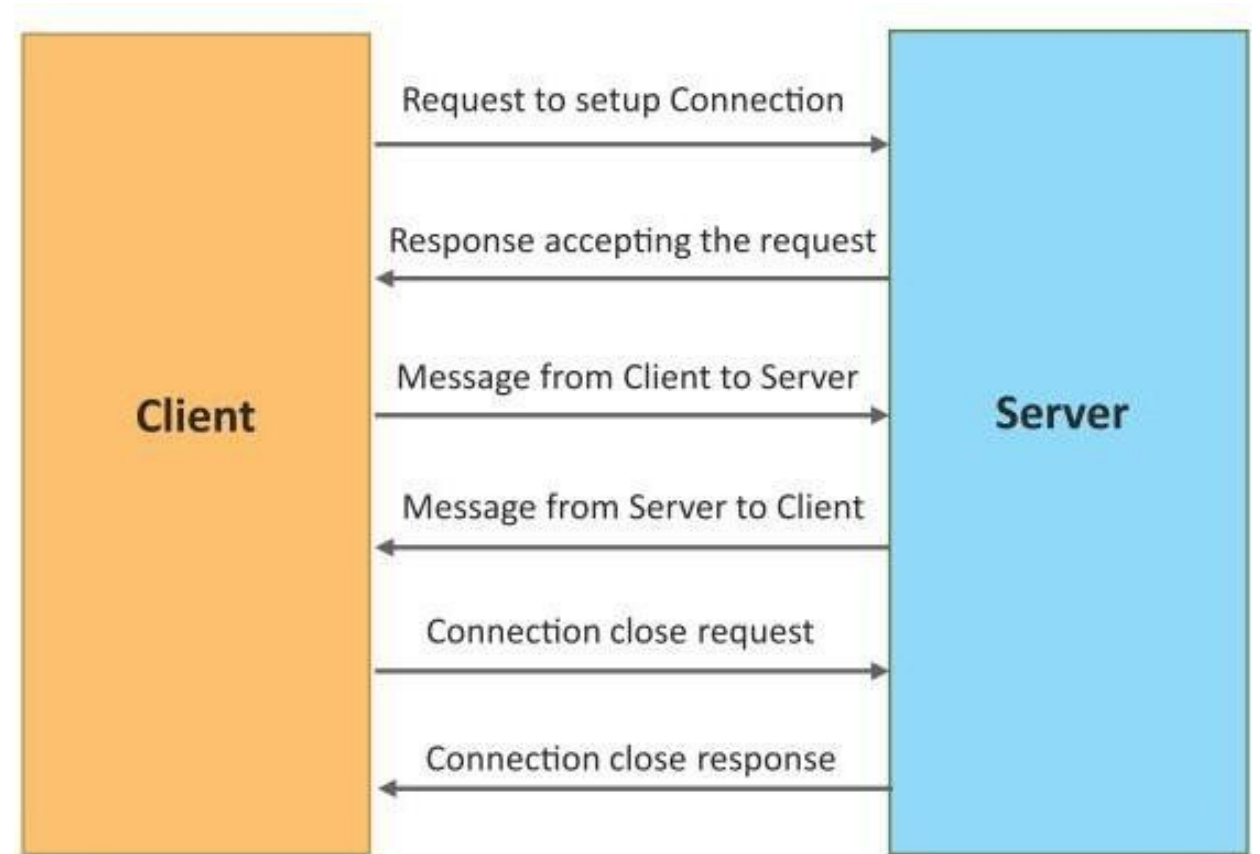
Push-Pull communication model

- Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers.
- Queues help in decoupling the messaging between the producers and consumers.
- Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data.



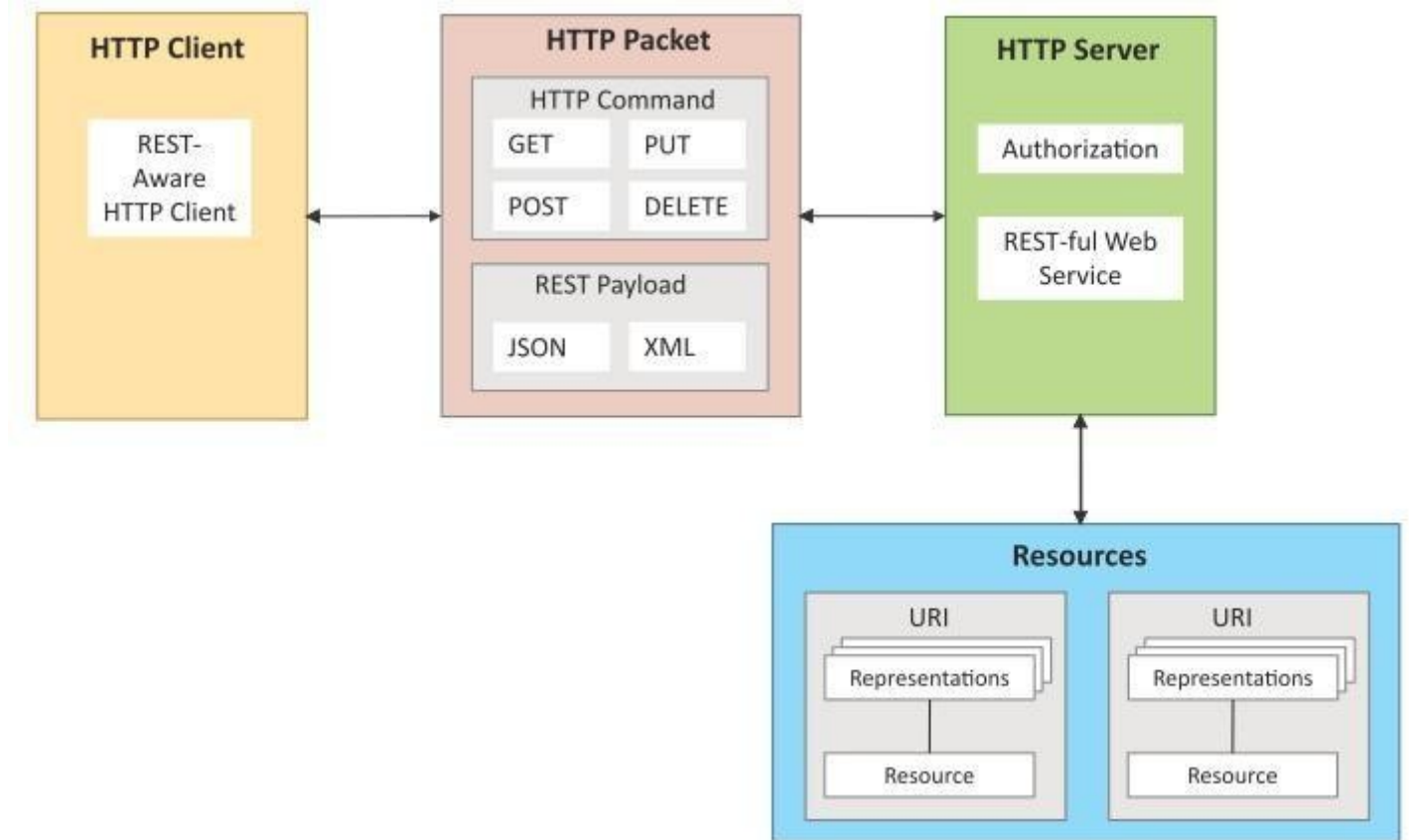
Exclusive Pair communication model

- Exclusive Pair is a bidirectional, fully duplex communication model that uses a persistent connection between the client and server.
- Once the connection is setup it remains open until the client sends a request to close the connection.
- Client and server can send messages to each other after connection setup.



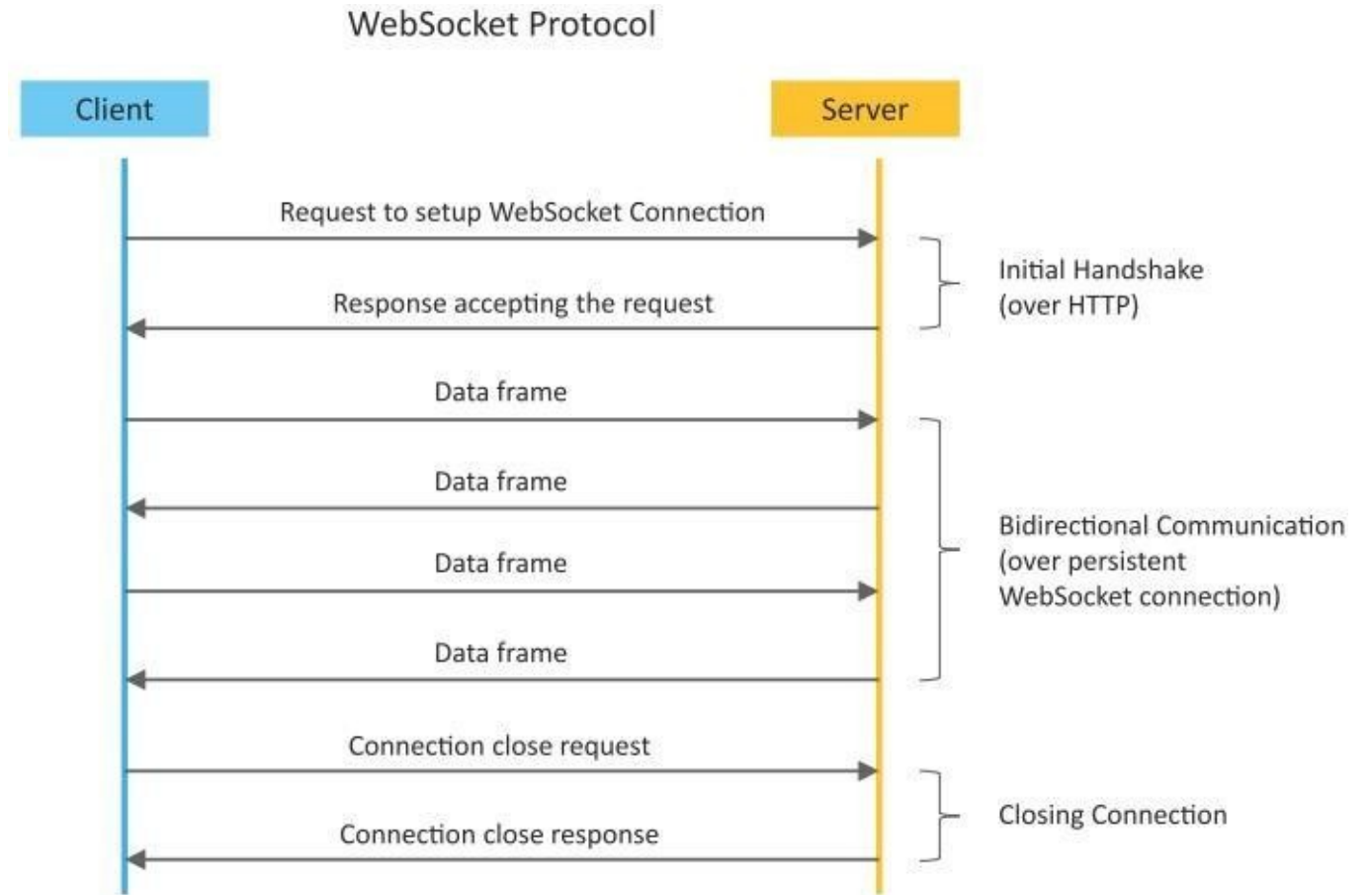
REST-based Communication APIs

- Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred.
- REST APIs follow the request-response communication model.
- The REST architectural constraints apply to the components, connectors, and data elements, within a distributed hypermedia system.



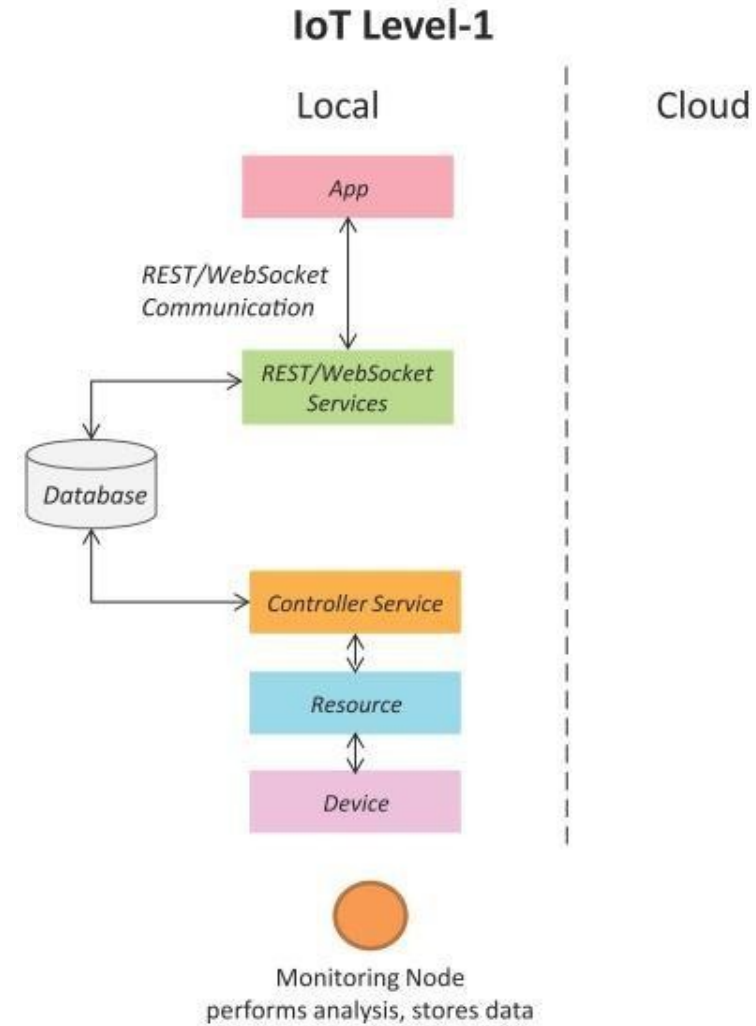
WebSocket-based Communication APIs

- WebSocket APIs allow bi-directional, full duplex communication between clients and servers.
- WebSocket APIs follow the **exclusive pair** communication model



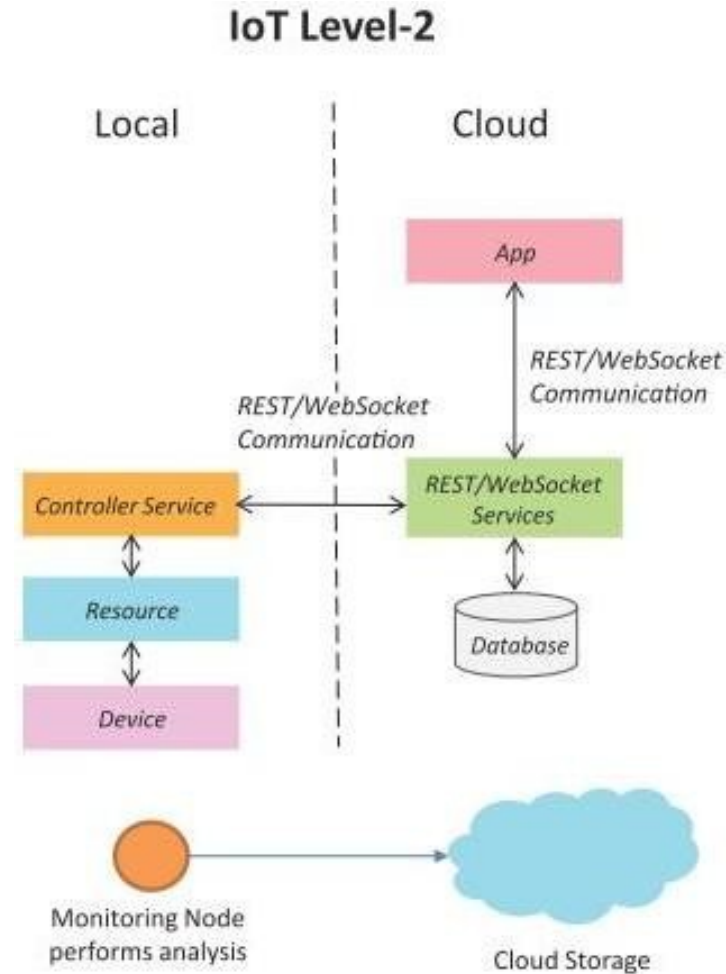
IoT Level-1

- A level-1 IoT system has a single node/device that performs sensing and/or actuation, stores data, performs analysis and hosts the application
- Level-1 IoT systems are suitable for modeling low-cost and low-complexity solutions where the data involved is not big and the analysis requirements are not computationally intensive.



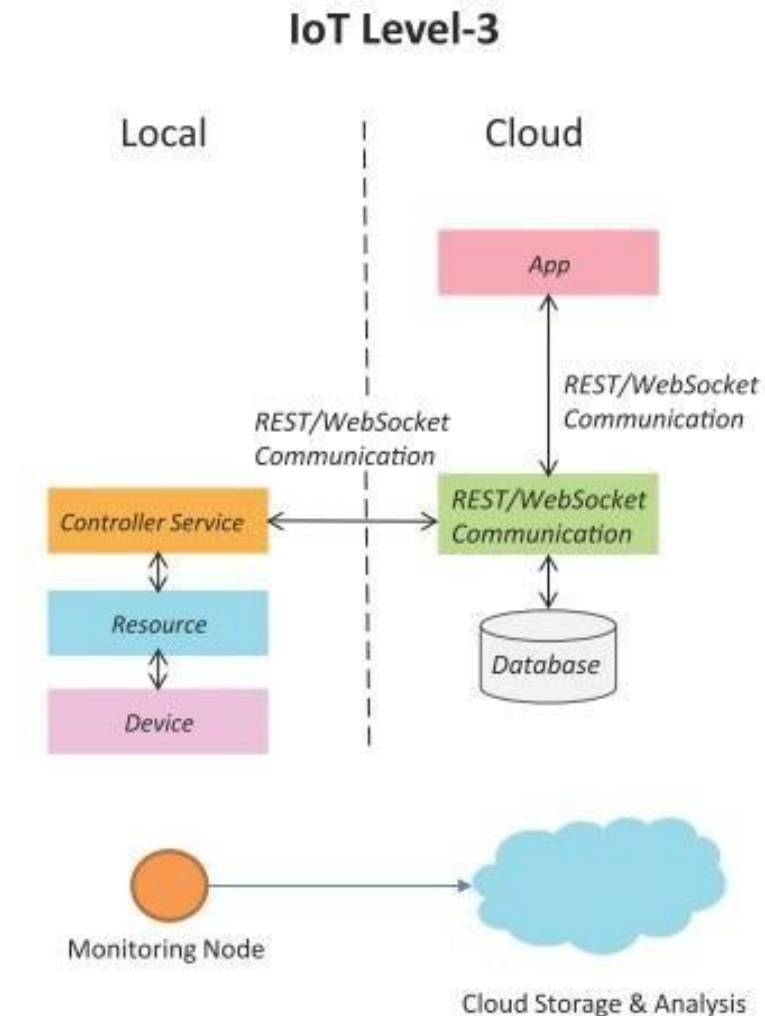
IoT Level-2

- A level-2 IoT system has a single node that performs sensing and/or actuation and local analysis.
- Data is stored in the cloud and application is usually cloud-based.
- Level-2 IoT systems are suitable for solutions where the data involved is big, however, the primary analysis requirement is not computationally intensive and can be done locally itself.



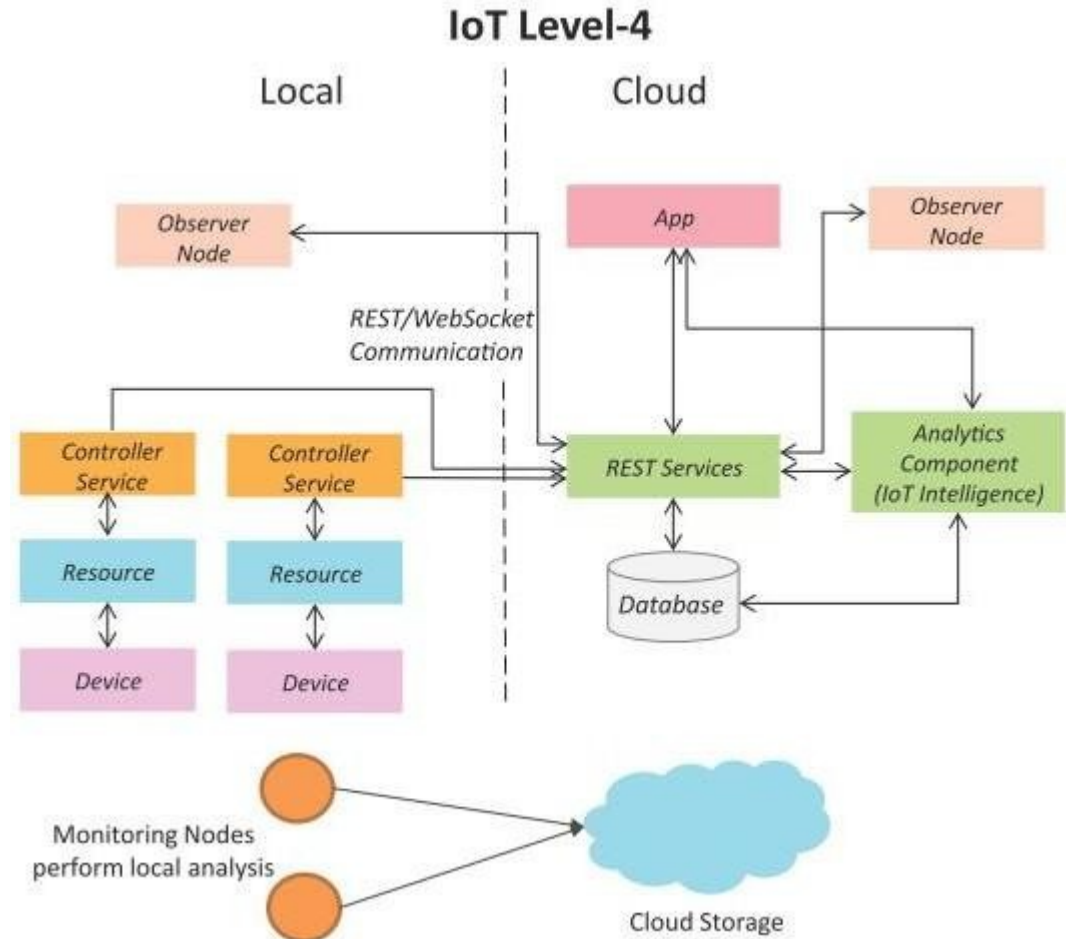
IoT Level-3

- A level-3 IoT system has a single node. Data is stored and analyzed in the cloud and application is cloud-based.
- Level-3 IoT systems are suitable for solutions where the data involved is big and the analysis requirements are computationally intensive.



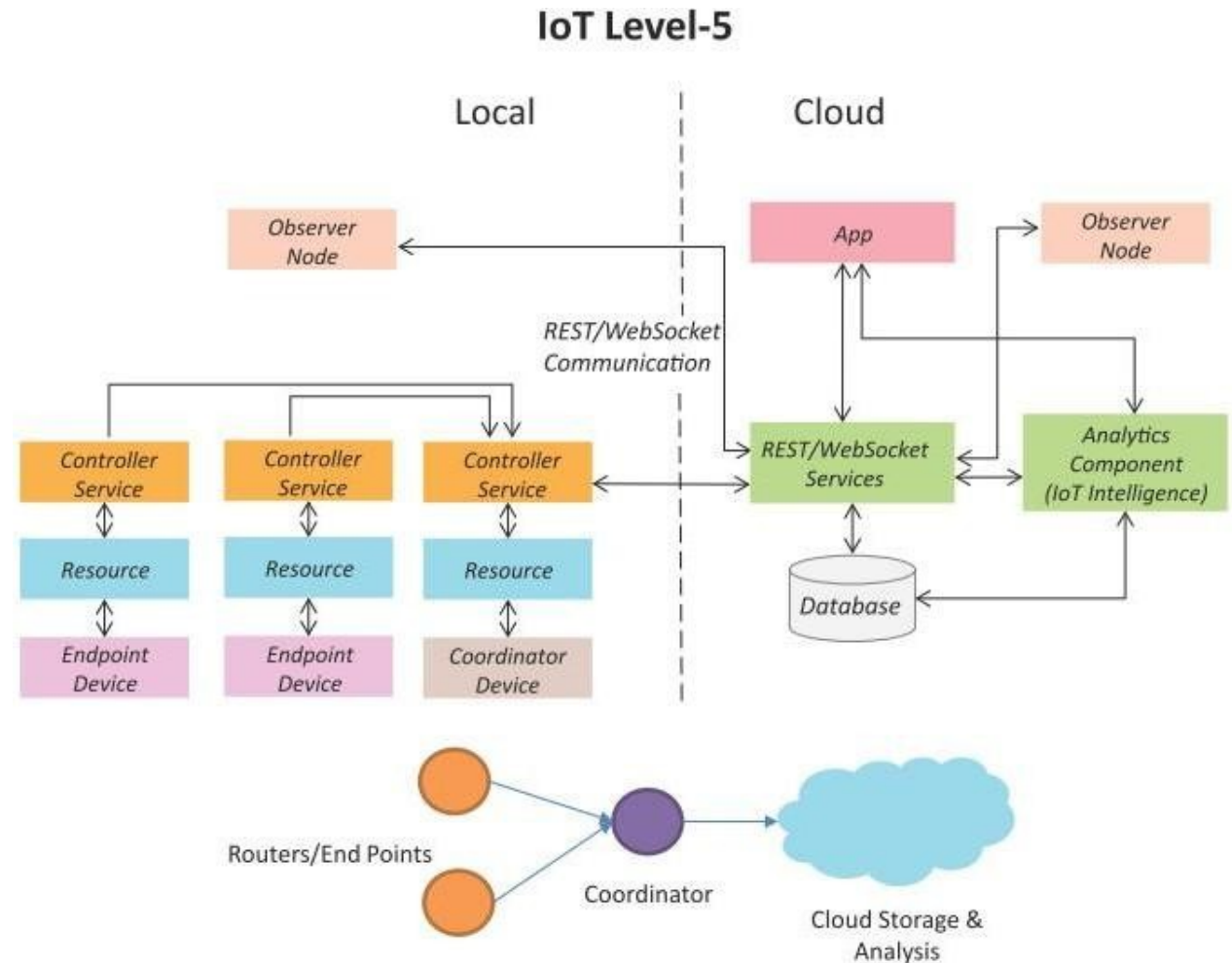
IoT Level-4

- A level-4 IoT system has multiple nodes that perform local analysis. Data is stored in the cloud and application is cloud-based.
- Level-4 contains local and cloud-based observer nodes which can subscribe to and receive information collected in the cloud from IoT devices.
- Level-4 IoT systems are suitable for solutions where multiple nodes are required, the data involved is big and the analysis requirements are computationally intensive.



IoT Level-5

- A level-5 IoT system has multiple end nodes and one coordinator node.
- The end nodes that perform sensing and/or actuation.
- Coordinator node collects data from the end nodes and sends to the cloud.
- Data is stored and analyzed in the cloud and application is cloud-based.
- Level-5 IoT systems are suitable for solutions based on wireless sensor networks, in which the data involved is big and the analysis requirements are computationally intensive.



IoT Level-6

- A level-6 IoT system has multiple independent end nodes that perform sensing and/or actuation and send data to the cloud.
- Data is stored in the cloud and application is cloud-based.
- The analytics component analyzes the data and stores the results in the cloud database.
- The results are visualized with the cloud-based application.
- The centralized controller is aware of the status of all the end nodes and sends control commands to the nodes.

